

Agenda Notebook

CS 101 - Catholic Polytechnic University - Spring 2026

Table of Contents

- [1. Opening and Closing Prayers](#)
 - [1.1. Opening Prayer](#)
 - [1.2. Closing Prayer](#)
 - [1.3. Why these prayers?](#)
- [2. DONE Week 1 - Course overview and first steps](#)
 - [2.1. Summary of the tool session](#)
- [3. DONE Week 2 - First assignment feedback & C++ tutorial](#)
 - [3.1. Housekeeping](#)
 - [3.2. Review: session 1](#)
 - [3.3. Bonus assignment "Program translator" \(Q&A\)](#)
 - [3.4. 10 super simple sample C++ programming problems](#)
 - [3.5. Notes](#)
 - [3.6. Summary - one thing you learnt today](#)
- [4. DONE Week 3 - C-style and C++ Arrays Part 1](#)
 - [4.1. Review: Quiz 1](#)
 - [4.2. Review: Quiz 2](#)
 - [4.3. Using AI, and time management for home assignments](#)
 - [4.4. Review: Programming assignment](#)
 - [4.5. Review: Introductory lecture](#)
 - [4.6. Arrays and vectors](#)
- [5. DONE Week 4 - C-style and C++ Arrays Part 2](#)
 - [5.1. Review: C-Style arrays](#)
 - [5.2. Assignment solution: Store and display product prices in an array](#)
- [6. DONE Week 5 - Arrays as function arguments and vectors](#)
 - [6.1. DONE What affects program performance](#)
 - [6.2. DONE Review: Arrays part 2](#)
 - [6.3. Lecture and codealong in Colab \(next week perhaps\)](#)
 - [6.4. Notes: Memory management in the computer](#)
 - [6.5. Lessons learnt / Take-aways](#)
- [7. DONE Week 6 - Vectors \(first encounter\)](#)
 - [7.1. From the Teams General Channel](#)
 - [7.1.1. Memory management in the computer](#)
 - [7.1.2. Coding in class](#)
 - [7.1.3. Visualizing program progress](#)
 - [7.2. Review: Functions and arrays - What's wrong with this code?](#)
- [8. Week 7 - Complexity & Pointers I](#)
 - [8.1. Chat: Python vs. Java vs. C++](#)

1. Opening and Closing Prayers

1.1. Opening Prayer

✠ *In the name of the Father, and of the Son, and of the Holy Spirit.*

**Our Father, who art in heaven,
hallowed by Thy name
Thy kingdom come,
Thy will be done on earth as it is in heaven.**

*Give us this day our daily bread
and forgive us our trespasses
as we forgive those who trespass against us.
And lead us not into temptation but deliver us from evil.
Amen.*

**Hail Mary, full of grace, the Lord is with thee.
Blessed art Thou among women,
and blessed is the fruit of Thy womb, Jesus.**

*Holy Mary, Mother of God, pray for us sinners,
now and at the hour of our death.
Amen*

St. Albert the Great

R: pray for us!

St. Thomas Aquinas,

R: pray for us!

St. Isidore of Seville,

R: pray for us!

**Glory be to the Father,
and to the Son,
and to the Holy Spirit,
as it was in the beginning,
is now, and ever shall be,
a world without end.
Amen.**

✠ *In the name of the Father, and of the Son, and of the Holy Spirit./*

1.2. Closing Prayer

✚ *In the name of the Father, and of the Son, and of the Holy Spirit.*

Come, Holy Spirit, Divine Creator,
true source of light and fountain of wisdom!

Pour forth Thy brilliance upon my dense intellect,
dissipate the darkness which covers me, that of sin and of ignorance.

Grant me a penetrating mind to understand,
a retentive memory, method and ease in learning,
the lucidity to comprehend, and abundant grace in expressing myself.

Guide the beginning of my work, direct its progress,
and bring it to successful completion.

This I ask through Jesus Christ,
true God and true man,
living and reigning with Thee and the Father,
forever and ever.

Amen. |

✚ *In the name of the Father, and of the Son, and of the Holy Spirit.*

1.3. Why these prayers?

- **The "Our Father"**, because we learn everything through him.
- **The "Hail Mary"**, because we need trust and humility to learn.
- **St. Albert the Great**, because he is the patron saint of scientists, philosophers, and students of the natural sciences.
- **St. Thomas Aquinas**, because he is the patron saint of Catholic schools, universities, and scholars.
- **St. Isidore of Seville**, because he is the patron saint of students, scholars, the internet, computer users and encyclopedias.
- **A student's prayer** is by St. Thomas Aquinas.

2. **DONE** Week 1 - Course overview and first steps

- [X] **Opening prayer** 5'
- [X] **Introductions**: Say a few words about yourself!
- [X] **Discussion**: Way of the course (review entry survey) - Times?

- [X] **Lecture:** [Course overview](#) (see also syllabus) 75' + 15'
- [X] **Practice:** Tooling (OneCompiler, Google Cloud Shell) 60'
- [X] **Dark horse tip:** Emacs + Org-mode (optional but useful).

2.1. Summary of the tool session

- You have to include `<iostream>` to print or get data.
- In C++, data can be streamed anywhere (using `std::cout`).
- OneCompiler.com is useful because it offers multiple languages.
- With Google Cloud Shell you can spin up a Linux VM fast.
- You should always ask students to fix other students' mistakes.

3. **DONE** Week 2 - First assignment feedback & C++ tutorial

- [] **Opening prayer** 5'
- [] **Housekeeping**
- [] **Practice:** 10 simple C++ vs. Java vs. Python programs
- [] **Asynchronous:** Watch remaining Java vs. C++ demos?
- [] **Assignment:** Finish remaining practice problems?
- [] **Bonus Assignment:** 10 more super simple C++ sample programs
- [] **Quiz 2:** MPC questions based on lecture and practice content
- [] **Session summary** (together)
- [] **Closing prayer** 5'

3.1. Housekeeping

I will always summarize the session and assignments on Saturday.

I will attempt to give you feedback during the week. Try me!

- [X] **Canvas:** Post-mortem message: Assignment/session summary
- [X] **Books** on C and C++
- [X] Further reading: **Linux** command-line, Literate programming
- [X] How **AI** use affects coding skills (of professionals)
- [X] The complete compiler chain - BPMN diagram (`gcc / g++`)
- [X] **Video:** How to compile "Hello, world!" on a Linux terminal
- [X] Reminder for quiz 1 (is an announcement/email better?)
- [X] Manager position for AI bot team (job advert)
- [X] "Occasional discomfort with being a product" (AI = machine)

3.2. Review: session 1

I put these together after looking at the **compilation practice** script.

1. What does `OneCompiler.com` not show?

The compilation of the program. It's an IDE that gives you the illusion that C++ is an interpreted language (like Python).

2. What does the command `./a.out` mean precisely?

To the **shell** program, it means to look for `a.out` in the current (`.`) directory if the current directory is not already in the `$PATH`.

3. Can you remember one true Linux command and what it means?

```
ls          # `ls` lists files
cd ..       # `cd` changes directory (.. -> next up)
```

```
0_overview.org
0_practice.org
1_intro_summary.org
1_java2cpp.org
2_introduction.org
3_arrays_class.org
3_arrays.html
3_arrays.org
3_complexity_class.org
3_complexity.org
4_pointers.org
agenda25.org
agenda_class.org
agenda.org
agenda.org_archive
announcements.html
announcements.org
architecture.org
assignment1.cpp
assignment.org
entry_pgm_solutions.org
example
fstream.cpp
hello
hello.cpp
input
java2cpp_class.org
quiz.org
syllabus.org
vectorFill.cpp
```

4. Does every C++ program have to have main? What is main?

Yes. main is a function and the entry point to the program proper, preceeded by the preprocessor directives (like #include).

5. What does void mean and where does it occur?

void as in main(void) means that no arguments will be expected. As a return value, void means "do not return any value."

3.3. Bonus assignment "Program translator" (Q&A)

You had to translate a Java (or a Python) program into C++. The program defined two variables and printed them using `std::cout`.

- Your question (paraphrased):

What is the `<<` doing?

- **Answer:** We'll discuss this later in class: `<<` is the stream insertion operator. It inserts data into the output stream (that streams data to any device, e.g. a screen), represented by `cout`, the standard output (`stdout`)

object.

So the chain printout command `cout << "Hello" << 42 << endl;`

1. inserts the string "Hello" into `cout`
2. inserts the integer 42 into `cout`
3. inserts `endl` (= newline + flush) into `cout`.

The idea of using a data stream makes data transfer (to the outside world) more general but also conceptually more difficult than just a C-style function like `printf`, or a Java function like `println`.

- Your question (paraphrased):

For the main program, I didn't use an argument like `String[] args`. Why don't I need that in C++? Or should I have included it?

- **Answer:** You don't need it in C++, and you also don't need it in Java unless you want to pass arguments on the command-line to the program.

The main function has an equivalent to the Java class argument `String[] args`. It is `main(int argc, char* argv[])`, and has two arguments that are actually easy to understand:

1. `argc` is the number of arguments fed to the program
2. `argv[]` is an array of strings that holds the arguments (an array of pointers to the beginning of each argument string) beginning with `argv[0]`, which is the name of the program, and continuing with as many arguments that are passed to `main`.

Background: Java is tightly coupled to the JVM, the system that is shipped with it and ensures its portability (but also its slowness). C++ is closer to the operating system, which simply starts your program and optionally provides arguments.

- A **code example** use with command-line arguments looks like this:

First the source code:

```
#include <iostream>
#include <string>

int main(int argc, char* argv[]) {
    // declare two strings
    std::string firstName, lastName;

    // print first command-line argument
    std::cout << "First name: " << argv[1] << std::endl
              << "Last name: " << argv[2] << std::endl;
}
```

The source code file `args.cpp` (tangled from the code cell above) is compiled, and then run on the command-line with two string arguments:

```
ls -l args.cpp # check if the file exists
g++ args.cpp -o args # compile and output executable `args`
./args Marcus Birkenkrahe
```

3.4. 10 super simple sample C++ programming problems

Topics to cover:

1. Variable declarations
2. Integer and floating-point arithmetic
3. Formatting output (cout, printf)
4. Handling user input (cin)
5. Variable type conversion (un/safe)
6. Conditionals (if, else, else if, switch)
7. Loops (aka **iterators** - for, while, do-while)
8. Arrays (aka **containers** of collections)
9. Functions (aka **transformations**)
10. Libraries (<iostream>, <string>, <iomanip>)

How we will cover them:

- Demo from me (Java + C++)
- You practice a simple example

What you will get later:

- Study sheet (PDF)
- Video if we don't finish today (likely)
- Bonus assignment: 10 more simple sample programs
- Quiz to test your understanding
- Assignment: submit ZIP file with all solutions

3.5. Notes

- [] Send NotebookLM demo script
- [] Send information on input/output streams
- [] More information on why programming languages for what
- [] Do a demo of pythontutor.com (online debugger)

3.6. Summary - one thing you learnt today

- printf requires specific format codes to print certain types: e.g. %i for int, and %f for float.
- Programming languages are created for a specific purposes, e.g. Python to do everything badly (but good enough mostly), or C++ to work with computer memory.
- C++ is a lot faster than all other languages (except C) - I'll send an illustration for that.
- The streaming operator << or >> depends on what you want to do: cin >> for input and cout << for output (like a pipe).
- All files are in github.com/birkenkrahe/cs101

4. **DONE** Week 3 - C-style and C++ Arrays Part 1

- [X] Opening prayer -
- [X] Review: quiz 1, quiz 2 - your input is needed!
- [] Video lecture - yay or nay?
- [] Using AI, time management for home assignments
- [] Review: programming assignment - feedback?
- [] Review: introductory lecture
- [] Lecture + code along + practice: Arrays and vectors
- [] Closing prayer -

4.1. Review: Quiz 1

- Which activity most directly relies on **abstraction**?

Choices were:

1. Turning a computer on and off
2. Typing source code into an editor
3. Installing an operating system
4. Using a data structure without knowing its internal implementation

In C++ a data structure with its operations and interface is called "Abstract Data Type" (ADT). Using a data structure without knowing its internal implementation directly relies on abstraction, because:

1. You use the public interface.
2. You do not depend on the implementation details.
3. The representation is hidden.

All four activities involve abstraction but only 4 directly relies on the abstraction boundary itself:

1. Turning a computer on/off: The power button abstracts voltage regulation, firmware bootstrapping, hardware initialization, kernel loading. You do not see what is happening electrically or at the BIOS/UEFI level.
2. Typing code in an editor: The editor abstracts memory buffers, file systems, rendering engines, input handling, OS calls.
3. Installing an operating system: This is extremely abstracted. You do not see partition tables, boot sectors, driver negotiation, kernel configuration, etc.

4. Which command compiles a C++ source file?

1. execute
2. g++
3. gcc-run
4. make

Of these, only g++ and make are commands. g++ is the compiler. make is a special program that requires a configuration file called Makefile, and it can, but does not have to compile a file.

5. Example:

```
echo -e '#include<iostream>\nint main(){std::cout<<"hello world"<<"\\n";return 0;}'
cat hello.cpp # view the program
g++ hello.cpp -o hello # compile & name output
ls hello* # check for files starting with `hello`
./hello # run it in the current (.) directory
```

```
#include<iostream>
int main(){std::cout<<"hello world"<<"\\n";return 0;}
hello
hello.cpp
hello world
```


4.2. Review: Quiz 2

What is the value of result?

```
double x = 3.5, y = 2.0, result = 0.0;
result = x * y;
```

- [X] 7.0
- [] 7
- [] 5.5
- []

0

```
double x = 3.5, y = 2.0, result = 0.0;
result = x * y;
std::cout << result << std::endl; // truncates by default to minimum
// number of non-zero decimals
std::cout << std::fixed << std::setprecision(1); // set precision explicitly
std::cout << result;
```

7
7.0

4.3. Using AI, and time management for home assignments

1. **Using AI:** Using AI for aid is perfectly fine. If you still have access to the conversation: you could ask the AI (in the same session) to give you a summary of the difficulties you encountered. If you bring these to class, it will give me something to work with - I'll know more about what you specifically need.
2. **Time management for home assignments:** As for the time - I promised you could cut off at 2 hours (without the AI though), and I stand by it. Anything that you didn't get done in that time is on me! I don't grade for mastery but for engagement & time put into solving a problem is a direct proof of engagement.
3. **AI and time management:** It is not guaranteed that using AI will save you time. In my experience, especially for new topics, it is the opposite, because you cannot correct the AI or see where it suggests the wrong route or just lies through its iron teeth. It is almost almost better for your time management if you spend some time **doing the work yourself** before turning to AI. Also: dependencies on newer models, and lack of personalization will cost you time.

4.4. Review: Programming assignment

- Let's revisit the solution & you tell me about your issues.
- **Problem:** Prompt user to enter their name and two integer numbers, print name, input numbers, their sum, difference, and product.
- **Sample output (exactly):**

```
Enter your name and two integers:
Hello Alice
a = 7, b = 3
sum = 10
```

```
diff = 4
prod = 21
```

- **Solution:**

```
// 1) declare variables: name, a, b, sum, diff, prod
string name;
int a=0,b=0,sum=0,diff=0,prod=0; // always initialize vars
// 2) print the prompt
cout << "Enter your name and two integers:" << endl;
// 3) read name, a, b
cin >> name >> a >> b; // using input stream object/operator cin >>
// 4) compute sum, diff, prod
sum = a + b;
diff = a - b;
prod = a * b;
// 5) print the formatted report
cout << "Hello " << name << "\n";
cout << "a = " << a << ", b = " << b << "\n";
cout << "sum = " << sum << endl
<< "diff = " << diff << endl
<< "prod = " << prod << endl;
```

```
Enter your name and two integers:
Hello
a = 0, b = 0
sum = 0
diff = 0
prod = 0
```

- Test the file assignment1.cpp on the command-line:

```
ls ass*.cpp # listing with wildcard *
g++ ass*.cpp -o example # compiling and naming output
ls -l example # check if the output file was produced
echo "Alice 7 3" | ./example # run executable with pipeline
```

```
assignment1.cpp
-rwxrwxr-x 1 aletheia aletheia 17440 Mar 11 06:53 example
Enter your name and two integers:
Hello Alice
a = 7, b = 3
sum = 10
diff = 4
prod = 21
```

- Check out <https://open.kattis.com> (competitive site)

4.5. Review: Introductory lecture

- What did you think of the recording (or the lecture)?
- What do you remember (from the recording/lecture)?
 1. Differences in data structures in different languages
- My summary (via ChatGPT who analysed the transcript):
 1. Learn concepts top-down, bottom-up, or by analogy.

2. Data structures organize and manage data.
 3. They enable storage, access, modification, and search.
 4. Data structures are everywhere in computing.
 5. Same structure, different names across disciplines.
 6. Arrays: fixed size collections, fixed compile-time.
 7. Vectors: dynamically sized containers, fixed at runtime.
 8. Data structure = noun; algorithm = verb.
 9. Scale changes what solutions are acceptable.
 10. Language choice affects available abstractions.
 11. Choose structure after understanding the problem.
 12. Design is planned and iterative, not random tinkering.
 13. Wrong structure can make problem unsolvable.
 14. Priority queues handle urgency better than FIFO/LIFO.
- Programming starts with modeling, not syntax (pseudocode).

4.6. Arrays and vectors

What did you learn (or remember) today?

1. auto
2. partial initialization with a list { . . . }
3. putting string values into an array

5. **DONE** Week 4 - C-style and C++ Arrays Part 2

- ☒ Feedback on the recording? [or lecture script]
- ☒ Feedback on quizzes? [one attempt only]
- ☒ Feedback on assignments?
- ☒ Feedback on the teaching speed?
- ☒ Feedback on your learning progress?
- ☐ Reference variables
- ☐ Reading input from files with `fstream`
- ☐ Converting index-based for loops to range-based for loops
- ☐ Processing array contents
- ☐ Assigning arrays
- ☐ Summing array elements
- ☐ Arrays as function arguments

5.1. Review: C-Style arrays

- We begin each session with a (mostly) short review of recent material.
- You should know at least some if not all answers given.
- If you don't, you should at least understand the answer.
- Being able to read code is an important, trainable skill.

1. What do we want to know about a data structure?

1. What is its purpose?
2. How are the data stored?
3. How do you define it?
4. How do you access it?
5. How do you copy it?

6. How do you extend it?
 7. How do you delete it?
 8. What else is worth knowing about it?
2. Can you answer these questions for C-style arrays?

1. What is the purpose of an array?

An array stores values of only **one** data type (e.g. double).

2. How are data in an array stored (in memory)?

An array stores its elements **contiguously** in memory, i.e. one memory cell directly on the next until the end is reached.

3. How do you declare and define an array?

```
// data type + name + size = value list
int array[5] {10, 20, 30, 40, 50};
```

4. How do you access an array?

```
int numbers[6] {0};
// loop over array indices
for (int i = 0; i < 6; i++)
    cout << numbers[i] << " ";
puts("");
// loop over the range of values
for (auto num : numbers)
    cout << num << " ";
```

```
0 0 0 0 0 0
0 0 0 0 0 0
```

If I leave out numbers[6] and write numbers[] instead (which is okay as long as I initialize it, will this affect performance?

The compiler substitutes the 6 and the executable (at run-time) will be exactly the same.

The preprocessor is another part of the compiler chain.

```
#define PI 3.14 // preprocessor

cout << PI; // statement
```

```
3.14
```

Test this yourself, take a hello.cpp and compile it with g++ hello.cpp -i and then look at the result.

Only two ways to improve program performance:

1. algorithm (logical changes)

2. memory (not possible by design in Java/Python)

5. How do you copy an array?

```
int numbers1[] {100,200,300}; // declare & define a 3-element array
int numbers2[3]; // must specify size - declare another 3-element array
//numbers2 = numbers1; // not allowed
for (int i=0; i<3; i++) // index-based for loop
    numbers2[i]=numbers1[i]; // element-wise copy
for (auto num : numbers2) // range-based for loop
    cout << num << " "; // print values
```

```
100 200 300
```

6. How do you extend an array?

You cannot extend an array at **run-time**, when the compiled program is executed, only at **compile-time** (in the source code). Alternative: use `std::vector`.

What happened in Y2K - did it have to do with arrays?

7. How do you delete an array?

An array is deleted automatically when it goes out of **scope**.

```
#include <iostream>
int main() {
    { // scope begins
        int num[3]={10,20,30}; // exists in memory now
        cout << num[0] << " " << num[1] << " " << num[2];
    } // scope ends
    cout << num[0] << " " << num[1] << " " << num[2]; // error: num is
                                                    // no longer defined
    return 0;
}
```

8. Where are arrays defined in C++?

C-style arrays are **built-in data structures**, no library is required. However, there is a special `std::array` container class, too, in the Standard Template Library (STL), which knows its size, and can be copied as a whole.

9. What else is worth knowing about arrays?

- Out of bounds errors:

```
int array[6] {10, 20, 30, 40, 50, 60};
// loop over array indices
for (int i = 0; i < 6; i--)
    cout << array[i] << " ";
cout << array[6]; // segmentation fault
```

```
10 -1 1391585258 29039 1392676696 23525 632537670 32765 262093696 0 0 23525 6
```

3. On which different levels does C++ work with arrays?

```
std::string names[] {"James","Tori","Sam","Aliana", "Marcus"}; // defining a string array
// Print strings using index [] operator
for (int i=0;i<5;i++) cout << names[i] << " "; puts(""); // index level

// Print string addresses using pointers to memory locations
for (int i=0;i<5;i++) cout << names << " "; puts(""); // address (memory) level

// Print array values using a range-based loop
for (std::string name : names) cout << name << " "; // value (element) level
```

```
James Tori Sam Aliana Marcus
0x7ffe3307a810 0x7ffe3307a810 0x7ffe3307a810 0x7ffe3307a810 0x7ffe3307a810
James Tori Sam Aliana Marcus
```

4. What is GNU make?

A smart program that builds executables.

Example: if you have a source code file `hello.cpp`, running `make hello` on the command-line will build the executable `hello`.

Demo: on the Linux command-line

```
touch hello.cpp
ls -l hello.cpp
make hello
./hello
```

```
-rw-rw-r-- 1 aletheia aletheia 72 Mar 11 06:53 hello.cpp
g++      hello.cpp  -o hello
hello world
```

5.2. Assignment solution: Store and display product prices in an array

I created a [notebook](#) for this assignment ([YouTube](#)).

- **Problem:** Change the program we completed in class to store and display product prices, so that it:
 1. Lets a user enter 5 product prices instead of hardcoding them in the program.
 2. Validates that each input is a positive number (greater than zero). If the test fails, print a warning message and abort gracefully using `std::exit(EXIT_FAILURE)` from `<cstdlib>`.
- **Starter program completed in class:**

```
// Store product prices in an array
// Input: None
```

```
// Author: Marcus Birkenkrahe

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    // Set SIZE of array
    const int SIZE = 5;
    // Define array `prices` and assign product prices
    double prices[SIZE] {10.0,100.0,45.99,30.00,200.00};
    // Set precision
    cout << fixed << setprecision(2);
    // Print prices on one line
    for (int i=0; i<SIZE; i++)
        cout << prices[i] << " ";
    return 0;
}
```

```
10.00 100.00 45.99 30.00 200.00
```

- **Solution:**

```
// Store product prices in an array with input validation
// Input: 5 product prices (double)
// Output: print prices with precision 2 on one line if input OK
//         abort gracefully if input not OK.
// Author: Marcus Birkenkrahe

#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

int main()
{
    // Set SIZE of array
    const int SIZE = 5;
    // Declare array `prices`.
    double prices[SIZE];
    // Ask user for input
    cout << "Enter " << SIZE << " product prices:\n";
    // Input the array elements
    cin >> prices[0] >> prices[1] >> prices[2] >> prices[3] >> prices[4];
    // Loop over array to validate elements are positive
    for (int i=0; i<SIZE; i++) {
        if (prices[i] <= 0) {
            cout << "ERROR: Product price must be positive. Abort.\n";
            std::exit(EXIT_FAILURE);
        }
    }
    // Set precision for output
    cout << fixed << setprecision(2);
    // Print prices on one line
    for (int i=0; i<SIZE; i++)
        cout << prices[i] << " ";
    puts("");
    return 0;
}
```

```
Enter 5 product prices:
0.00 0.00 0.00 0.00 0.00
```

- Test the program `prices.cpp` on the command-line:

```
ls -l prices.cpp
make prices
echo "10.00 100.00 45.99 30.00 200.00 " | ./prices
echo "10.00 100.00 45.99 30.00 0 " | ./prices
echo "10.00 100.00 -45.99 30.00 0 " | ./prices
```

- The best way to learn fast new ways is to learn to navigate cppreference.com.

6. **DONE** Week 5 - Arrays as function arguments and vectors

- [X] **Time check:** We are 1/3 through the term! Hoorah!
- [X] **This week:** Delayed grading & assignments (I'm traveling).
- [X] **Assignments deadlines:** Now Friday at midnight.
- [X] **Review:** What affects program performance?
- [] **Review:** Arrays part 2 - references and operators.
- [] **Lecture:** Arrays as function arguments, and vectors.

6.1. **DONE** What affects program performance

I said summarily that only algorithmic changes and memory manipulation affect program performance. This was perhaps a little too short. The whole story is a tad longer:

The hierarchy of run-time performance impact roughly looks like this:

1. **Algorithmic time complexity** (how long it takes to solve a problem step-by-step)
2. **Choice of data structure** (what container you pick for your numbers for example)
3. **Memory access patterns** (which locations are accessed, in what sequence, how often, and in which spatial proximity)
4. **Avoiding unnecessary copies** (especially important for functions)
5. **Compiler optimization friendliness** (for example program order, defining prototypes before the definition etc.)

What is correct: You cannot optimize performance by rearranging punctuation, and compile-time syntax differences (like `num[]` instead of `num[6]`) alone do not improve run-time.

You'll have a better understanding of this once we've covered time **complexity**, **pointers**, and other **container** types.

A good book on this efficiency stuff in C++ is the open content textbook by Pat Morin, "[Open Data Structures](#)". The gold standard is the fat tome of a book by Bryant/O'Halloran, which I also use in my computer architectures class, "[CS:APP](#)" (Computer systems: A Programmer's Perspective). A classic is Brooks' "[The Mythical Man-Month](#)" (Software Engineering and it stresses).

Donald Knuth, the inventor of "[Literate Programming](#)", said: "Premature optimization is the root of all evil." At this point in your journey, you should worry more about syntax and readability than performance.

6.2. **DONE** Review: Arrays part 2

1. What is a **reference** in C++?

A reference is an **alias** for an existing variable. Any operation on the reference affects the original variable.

2. What symbol is used to declare a reference variable?

The ampersand symbol: &, or the address-of operator. So you say &foo is "address-of foo", short for "memory address of foo".

3. Why must a reference be initialized when declared?

Because a reference must immediately refer to an existing variable and cannot remain unbound. **It's not a variable**, only looks like it.

```
int x=10;
int &ref;
```

4. Can a reference be changed to refer to another variable?

No. After initialization, a reference cannot be reseated to refer to a different variable.

```
int x, y;
int &ref = x;
int &ref = y;
```

5. Does a reference require additional storage?

No. A reference does not require extra storage. At run-time, it is optimized away entirely by the compiler.

6. Is the ampersand & symbol overloaded?

Yes. It is also used as a **logical/bitwise** operator, and to declaring references and **address-of** operations.

```
cout << boolalpha << ("red"=="blue" && 1 == 0);
```

```
false
```

7. In a range-based for loop, what does the loop variable receive by default?

A copy of each array element. If it changes, the original array is not changed.

8. Why must the loop variable be declared with & to modify elements?

Because only a reference allows direct modification of the original array elements (via the address-of the array elements).

9. What is the effect of ++score[2]?

It increments `score[2]` before printing or using its value. The increment is seen before the subscript operator is applied.

10. What is the effect of `score[4]++`?

It prints or uses the original value, then increments it. The subscript operator is seen before the increment is applied.

11. What does `value[count--]; do`?

It accesses `value[count]`, then decreases `count` by one. So for `value[1]`, this becomes `value[0]`.

12. What does `value[count]--; do`?

It decreases the value stored at index `count` by one. So if `value[1]` is 1, then `value[1]` becomes 0.

13. What does an **array name** represent in an expression?

It represents the **starting memory address** of the array.

14. Why does `array2 = array2;` not work to copy one array into another?

Because C-style arrays cannot be assigned directly; their starting memory address cannot be changed - the compiler sees this: `&array2[0] = &array1[0]`. That's like saying that the address for two physically different houses is identical.

15. What does `cout << num;` print for an `int` array?

It prints the starting memory address of the array.

```
int num[2] {100,200};  
cout << num << endl;  
cout << &num[0];
```

```
0x7fff7eb59a50  
0x7fff7eb59a50
```

16. Why is `&num[1]` four bytes ahead of `&num[0]`?

Because an `int` typically occupies 4 bytes in memory. "Typically", because the exact size depends on the computer architecture.

17. What is wrong with this program?

```
int x = 10;  
int &ref;  
ref = x;
```

The reference is not initialized when declared. A reference must immediately bind to an existing variable.

18. What is wrong with this program?

```
int a = 5;
int b = 10;
int &ref = a;
ref = b;
```

The reference does not reseat to b. Instead, the assignment stores the value of b into a, because ref is an alias for a.

19. What is wrong with this program?

```
int arr[3] = {1,2,3};
for (int x : arr) {
    x = x * 2;
}
```

The loop variable x is a copy of each element. The original array elements are not modified because no reference is used.

20. What is wrong with this program?

```
int a[3] = {1,2,3};
int b[3];
b = a;
```

C-style arrays cannot be assigned directly. The array name represents a fixed starting address that cannot be changed. Elements must be copied individually.

6.3. ~~Lecture and code along in Colab~~ (next week perhaps)

Today, we're going to experiment with literate programming: Instead of using OneCompiler.com, we're going to use Colab and you code along and solve problems in a Jupyter notebook instead of in an IDE.

6.4. Notes: Memory management in the computer

Following on from James' question: How exactly does the computer manage the memory when arrays are defined?

[This is stuff you'll learn in detail in the courses on operating systems [year 4] and computer architecture [year 2] both of which I'll probably be teaching.]

1. The most important fact: To the normal user, the computer maintains the illusion that memory (as in RAM, not hard disk persistent memory) is **infinite**.
2. In fact, behind the lines, the **kernel** of the operating system manages the memory allocation so that programs (or applications) that currently are not doing anything, must return their memory allocations until they've got work again.
3. Independent of this memory management for multiple applications, for every individual program, memory can only be allocated once: when you define an array a, it begins at an address &a[0], and every other array b defined afterwards, is at a different address, &b[0].
4. This is why arrays cannot be copied over just by name (b==a). The way you see this in the documentation (e.g. cppreference.com) is: "The name a is not a modifiable lvalue", or "arrays decay to pointers."
5. Preview: a vector can be copied because it is a class and has a "copy constructor", and a "copy assignment operator" - functions that basically do the element-by-element copying without you seeing it in

a very efficient way (it is "cache-friendly", reallocates infrequently, and exploits bulk memory operations - "all-in-one").

[Copied to the Teams-General channel]

6.5. Lessons learnt / Take-aways

- MB: I need to start with simpler examples. And pay even more attention to memory management questions.
- How to use function to pass and loop over (C-style) arrays
- How to write functions efficiently in C++ (declaring/defining/using)
- I learnt the functions of functions & how to use it with containers

7. **DONE** Week 6 - Vectors (first encounter)

- [X] Can you confirm: End of term = May 8. Summer term = May 18?
- [X] Review: Functions and arrays ("What's wrong with this code?")
- [X] Next assignment: Inventory management with vectors
- [X] C++ vectors: `size()`, `push_back`, `pop_back()`, `clear()`, `empty()`
- [X] Next lecture: Time and space complexity (Big-O) [online lecture]
- [X] Next lecture + practice: Pointers (memory manipulation)

7.1. From the Teams General Channel

7.1.1. Memory management in the computer

Following on from James' question: How exactly does the computer manage the memory when arrays are defined?

[This is stuff you'll learn in detail in the courses on operating systems and computer architecture.]

1. The most important fact: To the normal user, the computer maintains the illusion that memory (as in RAM, not hard disk persistent memory) is **infinite**.
2. In fact, behind the lines, the **kernel** of the operating system manages the memory allocation so that programs (or applications) that currently are not doing anything, must return their memory allocations until they've got work again.
3. Independent of this memory management for multiple applications, for every individual program, memory can only be allocated once: when you define an array `a`, it begins at an address `&a[0]`, and every other array `b` defined afterwards, is at a different address, `&b[0]`.
4. This is why arrays cannot be copied over just by name (`b==a`). The way you see this in the documentation (e.g. cppreference.com) is: "The name `a` is not a modifiable lvalue", or "arrays decay to pointers."
5. Preview: a vector can be copied because it is a `class` and has a "copy constructor", and a "copy assignment operator" - functions that basically do the element-by-element copying without you seeing it in a very efficient way (it is "cache-friendly", reallocates infrequently, and exploits bulk memory operations - "all-in-one").

7.1.2. Coding in class

What if you're struggling to code in class on command as it were? Not worry for several reasons:

- There are three important skills related to programming: being able to **read** and understand programs, being able to **fix** programs, and being able to **write** programs.

- Of these, being able to write programs may be the least important, especially in an AI-infested (or -blessed?) future when **AI tools** will write most of the code.
- To learn to read and fix programs, you need to **practice** a lot of reading and fixing but you don't need to be able to program competitively (I am not very good at it myself).
- The assignments, for which you should take all the time you need, look stuff up, wrestle with etc. are the best way to train what you need to learn right now. Everything else is icing on the cake.
- So when I ask "are you done?", "do you need more time?" etc. don't worry if you cannot get the finished code to run. Instead, focus on understanding and on where your difficulties may lie, and ask.
- Apart from that, if you want to drill **competitive** programming, go to open.kattis.com and begin to solve easy problems. I even offered a course like that last term, and it was actually fun.
- For further reading and training, take a look at the competitive programmers handbook - [free PDF here](#).

7.1.3. Visualizing program progress

In the last session, we looked at pythontutor.com to visualize walking through a program. The dashboard showed the variables appearing on the "stack" (when we work with dynamic structures like the vector, they will appear on the "heap"), how the program moves into the function scope and out of it again, creating and deleting memory as it proceeds.

Pythontutor is excellent for small programs to identify logical errors - problems with your algorithm. For large programs, and for a lot more information, you need a professional debugging tool like gdb. While I would run it on the command-line (e.g. in Google Cloud Shell or Google Colab), there is an online version of gdb, onlinegdb.com (with a [tutorial](#)). In the course, we'll use gdb soon to give you an idea on how to do it.

7.2. Review: Functions and arrays - What's wrong with this code?

1. What's wrong with this code?

```
#include <iostream>
int add(int a, int b)
{
    std::cout << a + b << std::endl;
}
int main() {return 0;}
```

The function is declared to return int, but it does not return a value. A return statement such as `return a + b;` is required.

2. What's wrong with this code?

```
#include <iostream>
int main()
{
    std::cout << add(5,12);
    return 0;
}
```

The function `add` is used before it is declared or defined. A function prototype like `int add(int, int);` must appear before `main`.

3. What's wrong with this code?

```
void showValues(int num[])
{
    for (int i = 0; i < num; i++)
        cout << num[i] << endl;
}
```

num is a pointer to the first element of the array, not the array size. The function must receive the size as a separate parameter.

4. What's wrong with this code?

```
#include <iostream>

void showValues(int num[])
{
    for (int val : num)
        std::cout << val << std::endl;
}

int main() {return 0;}
```

The parameter num is treated as a pointer, not a full array. A range-based loop cannot determine the beginning and end of the range.

5. What's wrong with this code?

```
#include <iostream>

void showValues(const int num[], int size)
{
    for (int i = 1; i <= size; i++)
        std::cout << num[i] << std::endl;
}

int main()
{
    const int SIZE=4;
    int arr[SIZ] {100,200,300,400};
    showValues(arr,SIZ);
    return 0;
}
```

Array indexing starts at 0. The loop accesses num[size], which is outside the array bounds. The correct range is i = 0 to i < SIZE.

8. Week 7 - Complexity & Pointers I

- [] Chat: Python vs. Java vs. C++
- [] Time and Space Complexity
- [] Starting with pointers

8.1. Chat: Python vs. Java vs. C++

Believe it or not - I am finally learning Java properly - because in my new job, I was told, all intro courses use Java. I'm using the book "Algorithms" by Wayne and Sedgewick ([see here](#)), which is of intermediate difficulty. I

was interested in listing the differences between Python, C++ and Java, and especially AI's recommendation for the beginner courses, and it confirmed my choice (C++) for the intermediate course (CS 101). For algorithms, AI recommended to switch to Python (there is a nice book on this: [Dive into algorithms](#), by Tuckfield, NoStarch 2021).

I am inclined to follow the AI recommendation for courses I+II but to revert to Java for III because the argument of the Princeton book (from 2011) convinces me: Java provides a balance between **abstraction** (Python) and **control** (C++) that fits algorithm instruction.

Here is the breakdown for what it's worth:

1. First programming course (Introduction to programming)

Language	Pros (First Programming Course)	Cons
Python	Very simple syntax; quick results; minimal boilerplate; large ecosystem	Hides types and memory; transition to systems hard
Java	Strong typing; structured programs; clear OOP model; widely documented	Verbose syntax; slower feedback for beginners
C++	Close to hardware; good foundation for systems programming	Complex syntax; pointers; steep learning curve

ChatGPT recommends: Python.

2. Data structures (Intermediate programming)

Language	Pros	Cons
Python	Simple syntax; fast coding; widely known; good for prototyping ideas	Built-ins hide structures; dynamic typing; slower
Java	Strong typing; clean OOP; stable APIs; memory mgmt simpler than C++	Verbose; limited control over memory; less used now
C++	High performance; templates; STL; explicit memory model; close to hw	Complex language; harder for beginners; more bugs

ChatGPT recommends: C++

3. Algorithms course

Language	Pros (Algorithms Course)	Cons
Python	Rapid experimentation; concise code easy visualization and notebooks	Built-ins hide structures; slower for large inputs
Java	Strong typing; good ADT design; stable libraries; predictable perf.	Verbose; less control over memory/layout

Language	Pros (Algorithms Course)	Cons
C++	High performance; templates; STL; explicit memory model	Complex language features; harder for weaker students

ChatGPT recommends: Python

Author: Marcus Birkenkrahe

Created: 2026-03-11 Wed 06:53